Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

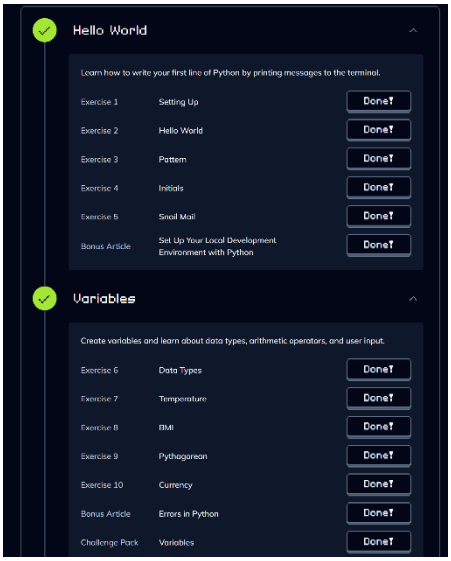
Tema: ” Learn Python ”

A efectuat : st. gr.SI – 262 Maister Darius

A verificat : asistent universitar Marius Dumitrașcu

Chișinău – 2026

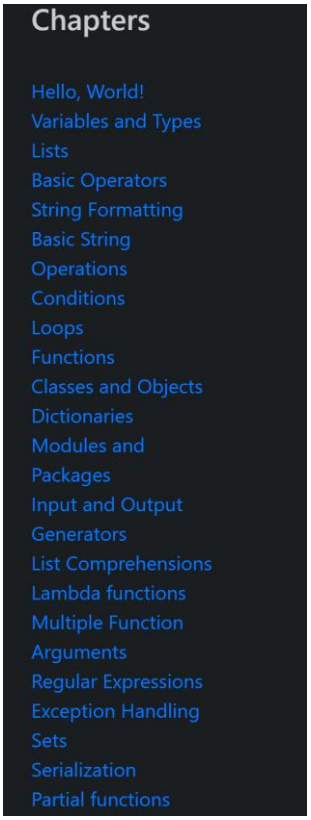
SARCINA LUCRĂRII



Prima sarcină a fost să completez cele 10 exercitii de bază de pe site-ul https://www.codedex.io/python.

Au fost niște exerciții simple bazate pe printare si lucrul cu variabile.

Ulterior am avut sarcina să studiez și să îndeplinesc sarcini practice la următoarele capitole de pe site-ul https://www.learnpython.org/.



script.py

```
1 print("Hello, World!")
```

IPython Shell

```
<script.py> output:
    Hello, World!

In [1]:
```

Great job!

Solution Submit

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

script.py

```
1  # change this code
2  mystring = "hello"
3  myfloat = float(10)
4  myint = 20
5
6  # testing code
7  if mystring == "hello":
8      print("String: %s" % mystring)
9  if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

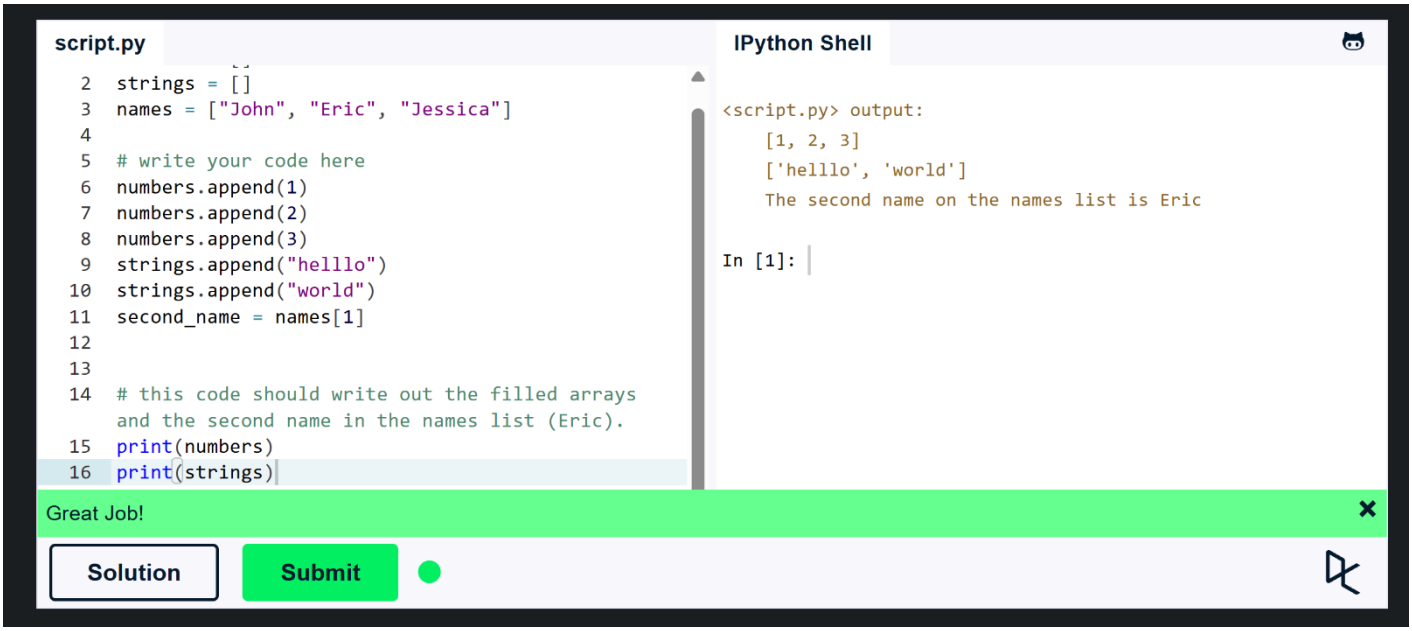
IPython Shell

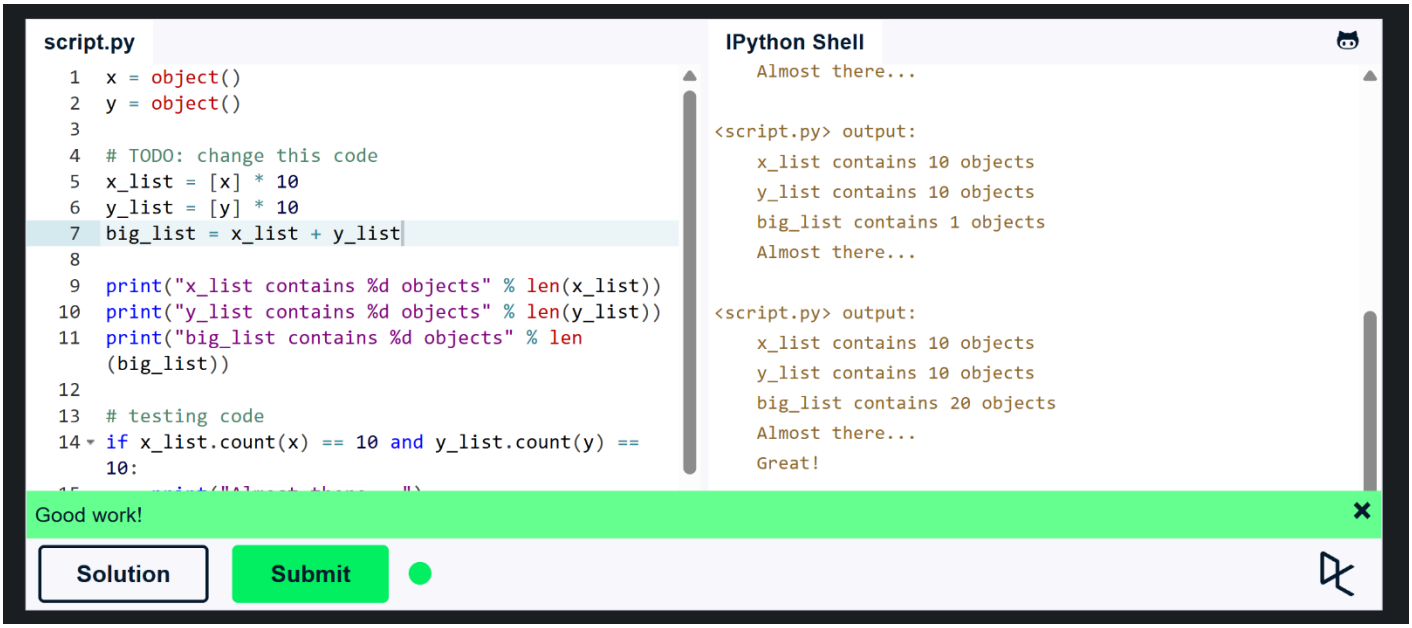
```
<script.py> output:
    String: hello
    Float: 10.000000
    Integer: 20

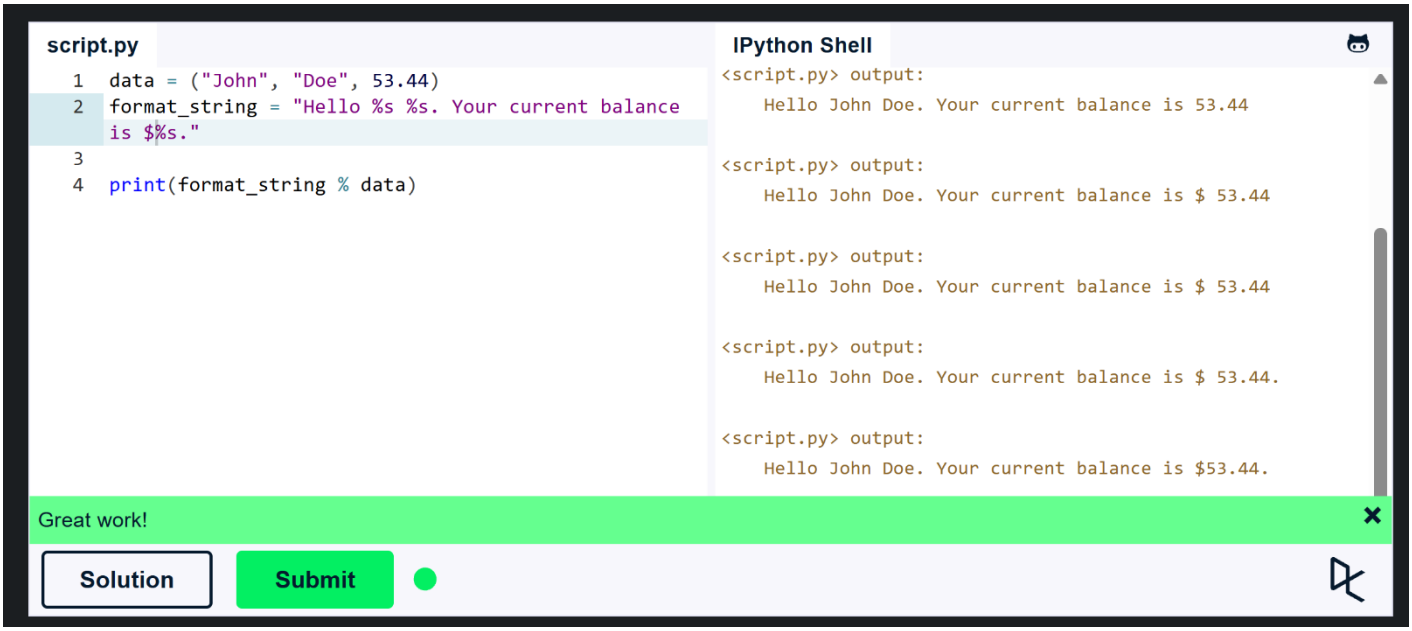
In [1]:
```

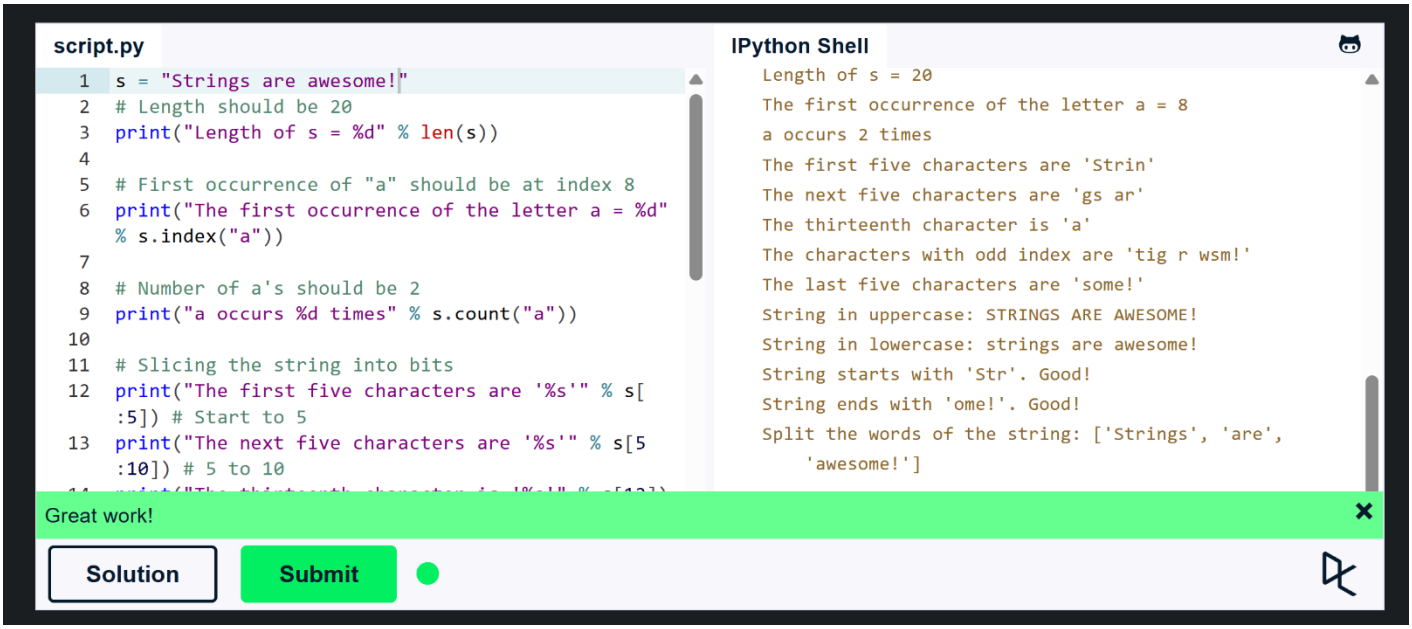
Great job!

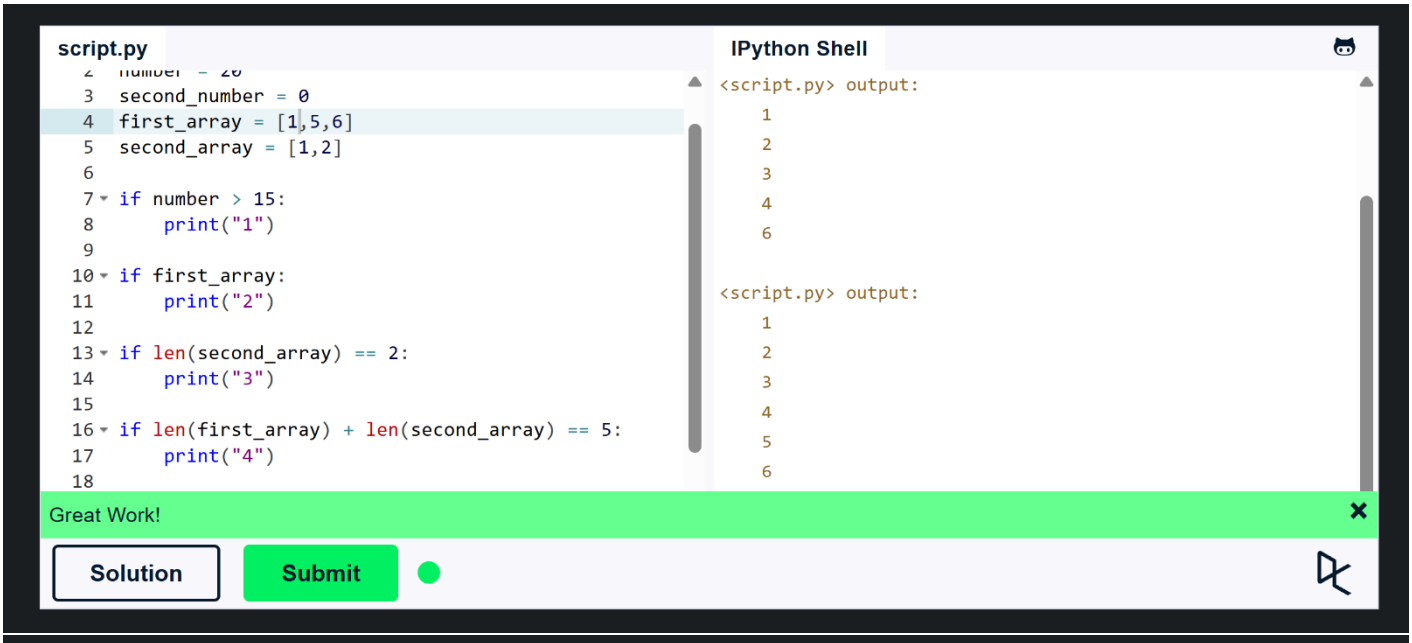
Solution Submit

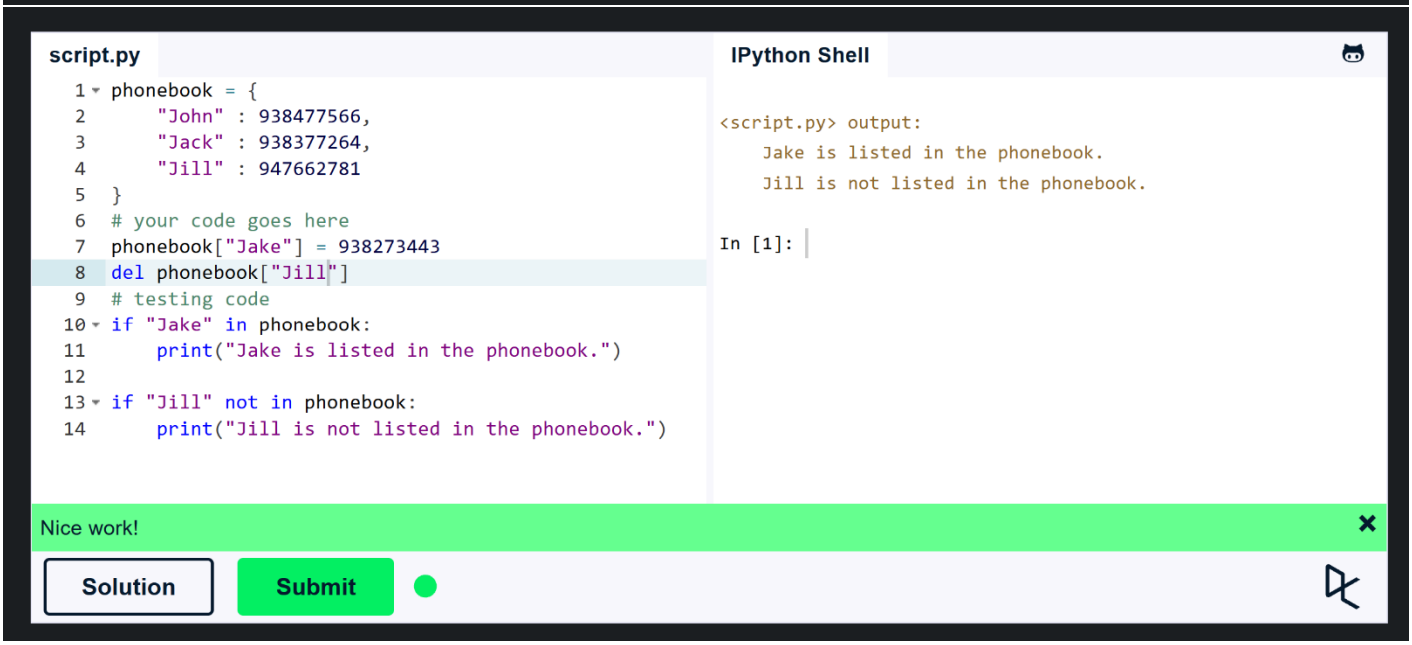

script.py
strings = []
names = ["John", "Eric", "Jessica"]
write your code here
numbers.append(1)
numbers.append(2)
numbers.append(3)
strings.append("helllo")
strings.append("world")
second_name = names[1]
this code should write out the filled arrays and the second name in the names list (Eric).
print(numbers)
print(strings)
IPython Shell
<script.py> output:
[1, 2, 3]
['helllo', 'world']
The second name on the names list is Eric
In [1]:
Great Job!
Solution
Submit

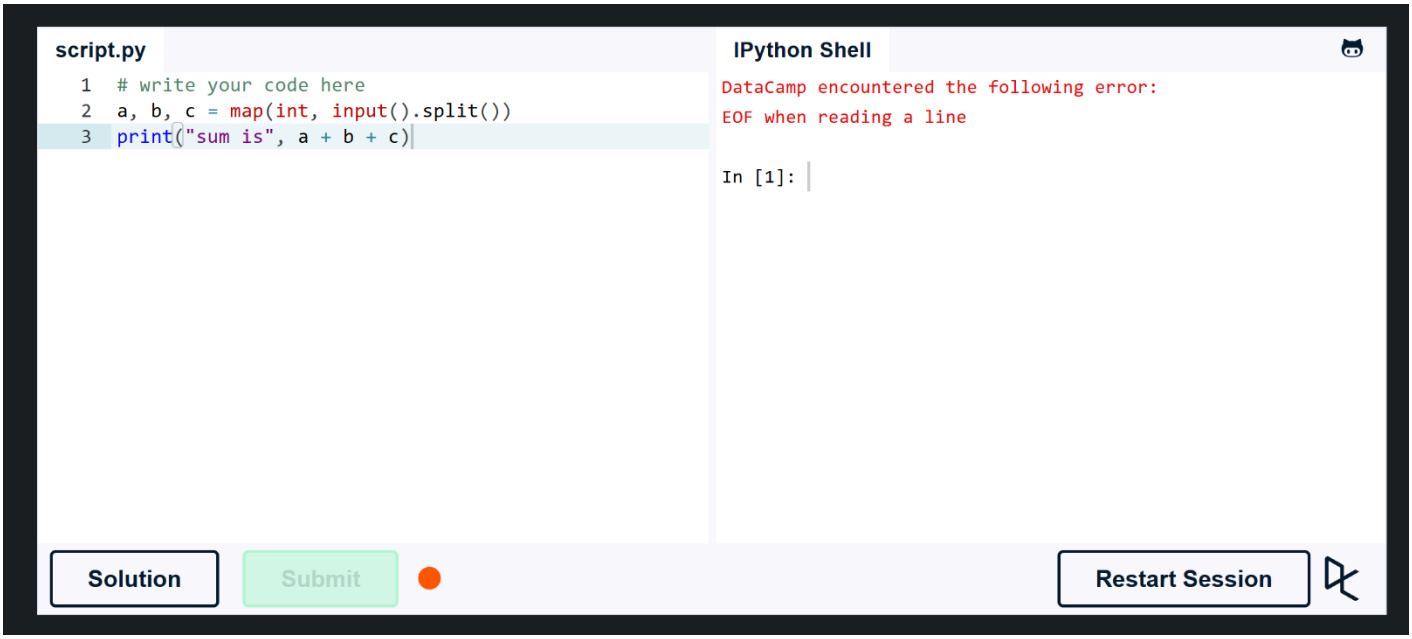

script.py
x = object()
y = object()
TODO: change this code
x_list = [x] * 10
y_list = [y] * 10
big_list = x_list + y_list
print("x_list contains %d objects" % len(x_list))
print("y_list contains %d objects" % len(y_list))
print("big_list contains %d objects" % len(big_list))
testing code
if x_list.count(x) == 10 and y_list.count(y) == 10:
IPython Shell
Almost there...
<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 1 objects
Almost there...
<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
Almost there...
Great!
Good work!
Solution
Submit

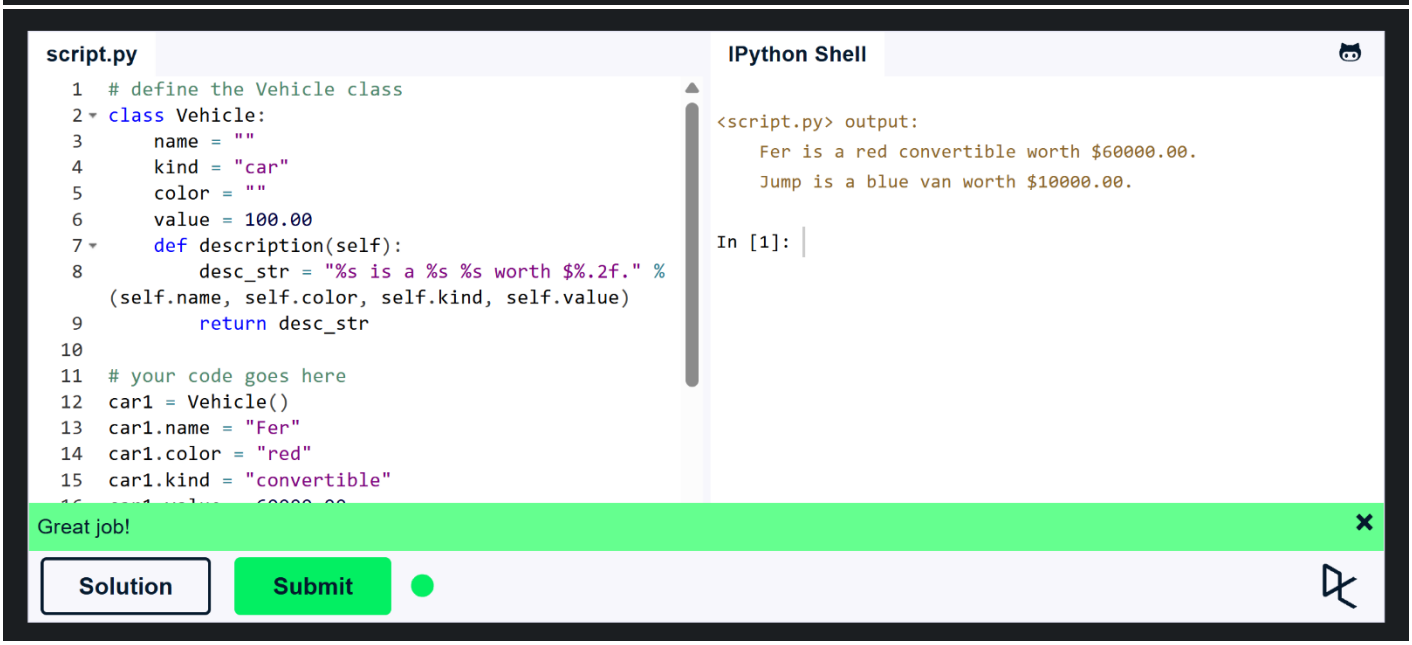

script.py
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance is $%s."
3
4 print(format_string % data)
IPython Shell
<script.py> output:
Hello John Doe. Your current balance is 53.44
<script.py> output:
Hello John Doe. Your current balance is $ 53.44
<script.py> output:
Hello John Doe. Your current balance is $ 53.44
<script.py> output:
Hello John Doe. Your current balance is $ 53.44.
<script.py> output:
Hello John Doe. Your current balance is $53.44.
Great work!
Solution
Submit

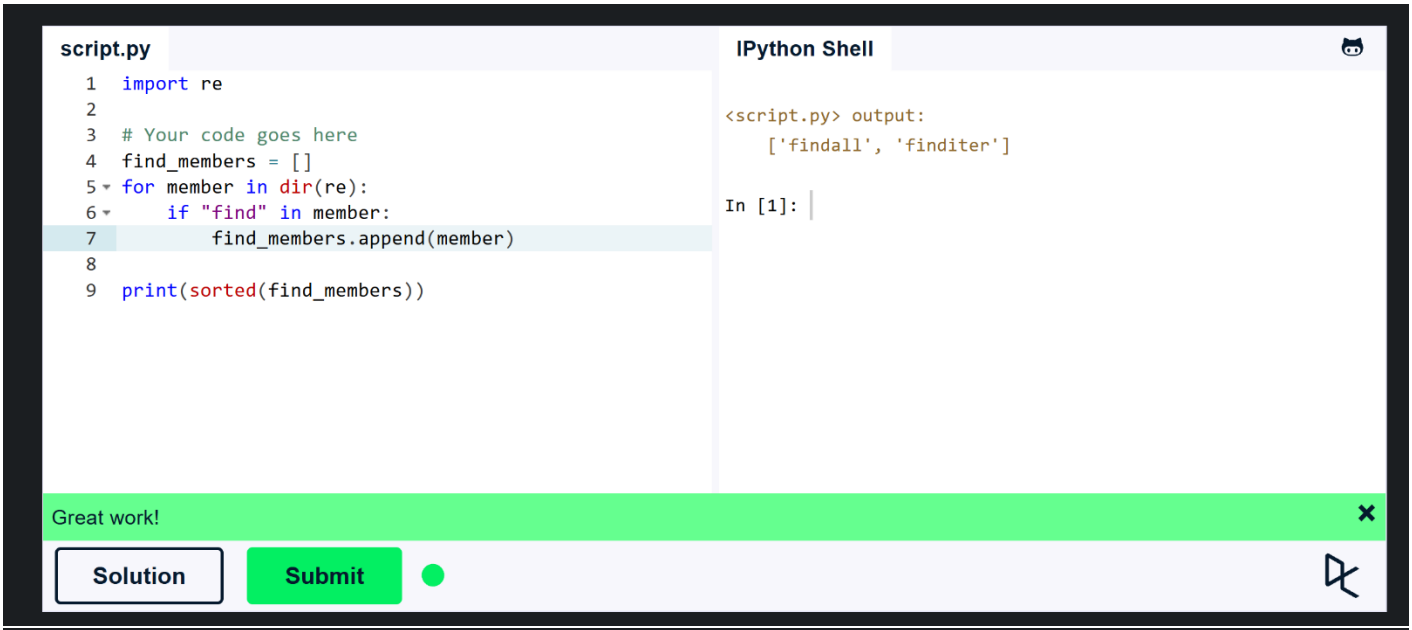

script.py
1 s = "Strings are awesome!"
2 # Length should be 20
3 print("Length of s = %d" % len(s))
4
5 # First occurrence of "a" should be at index 8
6 print("The first occurrence of the letter a = %d" % s.index("a"))
7
8 # Number of a's should be 2
9 print("a occurs %d times" % s.count("a"))
10
11 # Slicing the string into bits
12 print("The first five characters are '%s'" % s[:5]) # Start to 5
13 print("The next five characters are '%s'" % s[5:10]) # 5 to 10
IPython Shell
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are', 'awesome!']
Great work!
Solution
Submit

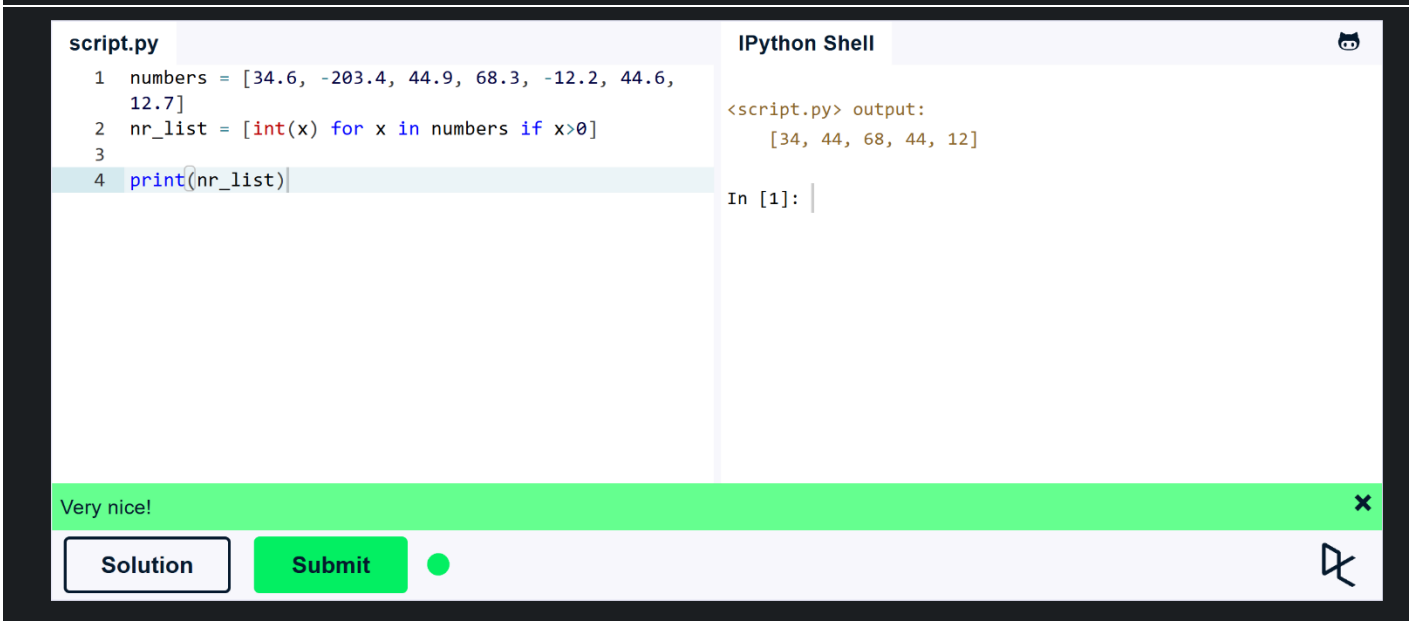

script.py
second_number = 0
first_array = [1,5,6]
second_array = [1,2]
if number > 15:
print("1")
if first_array:
print("2")
if len(second_array) == 2:
print("3")
if len(first_array) + len(second_array) == 5:
print("4")
IPython Shell
<script.py> output:
1
2
3
4
6
<script.py> output:
1
2
3
4
5
6
Great Work!
Solution
Submit

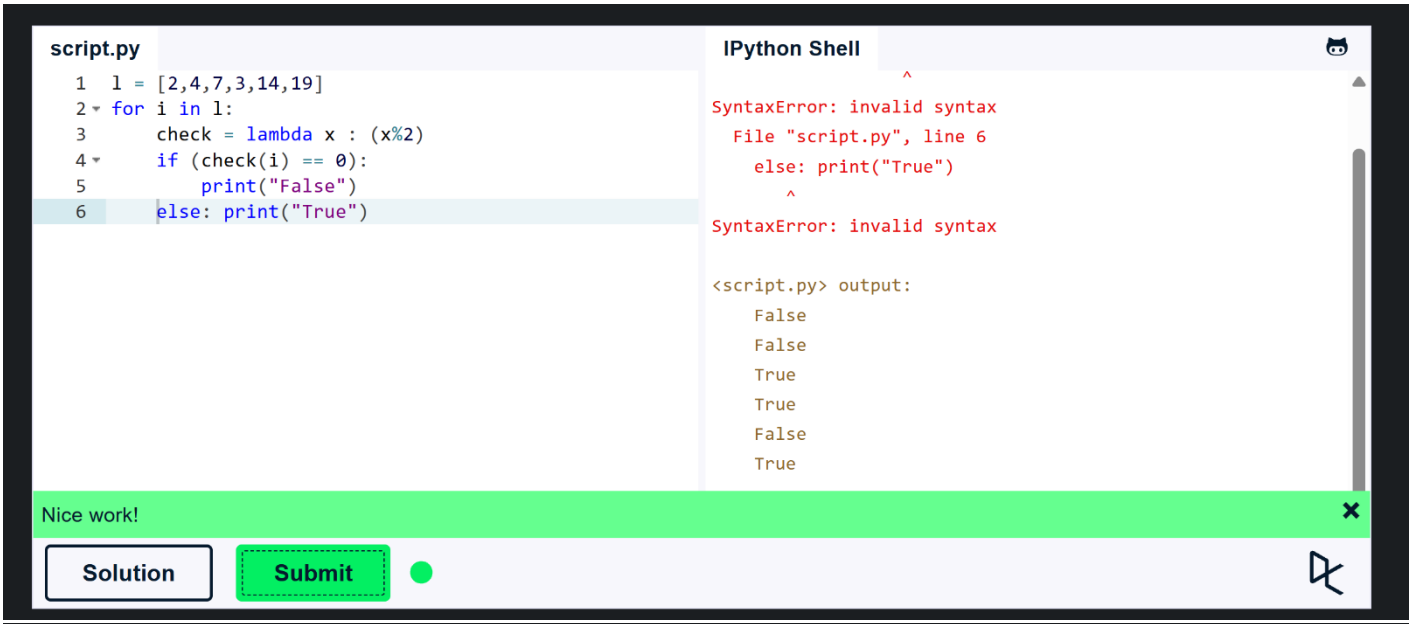

script.py
phonebook = {
"John" : 938477566,
"Jack" : 938377264,
"Jill" : 947662781
}
your code goes here
phonebook["Jake"] = 938273443
del phonebook["Jill"]
testing code
if "Jake" in phonebook:
print("Jake is listed in the phonebook.")
if "Jill" not in phonebook:
print("Jill is not listed in the phonebook.")
IPython Shell
<script.py> output:
Jake is listed in the phonebook.
Jill is not listed in the phonebook.
In [1]:
Nice work!
Solution
Submit

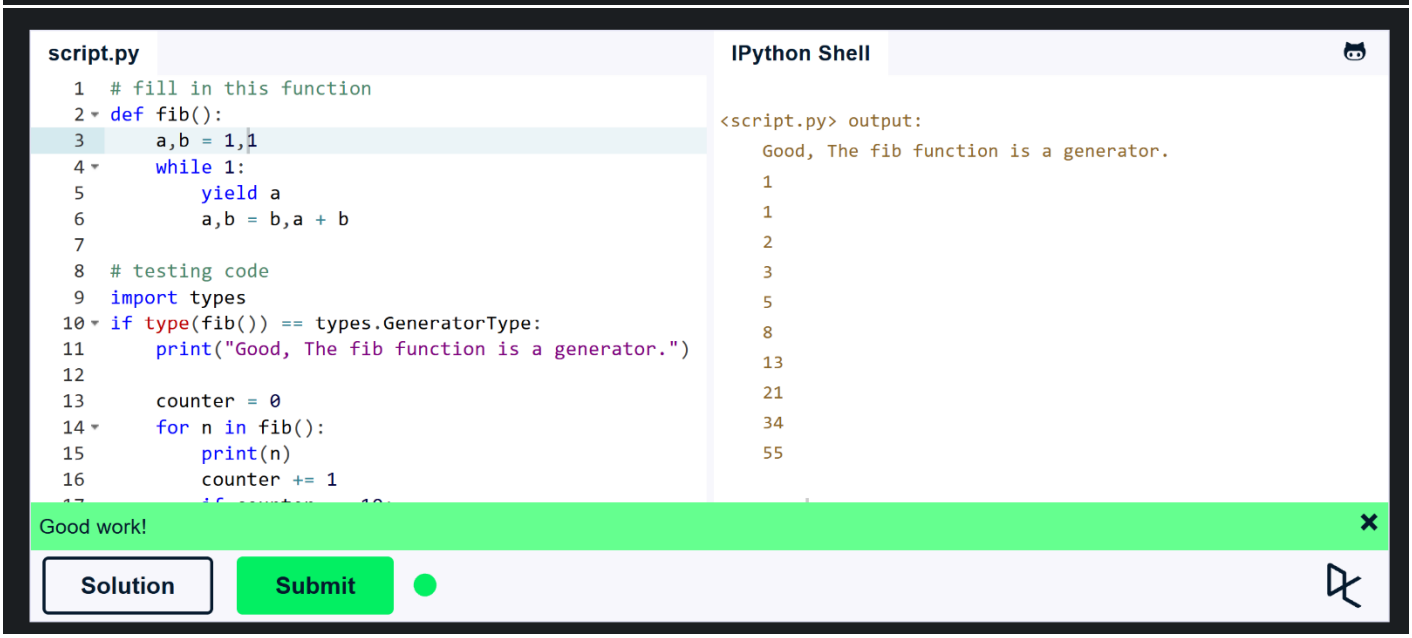

script.py
1 # write your code here
2 a, b, c = map(int, input().split())
3 print("sum is", a + b + c)
IPython Shell
DataCamp encountered the following error:
EOF when reading a line
In [1]:
Solution
Submit
Restart Session

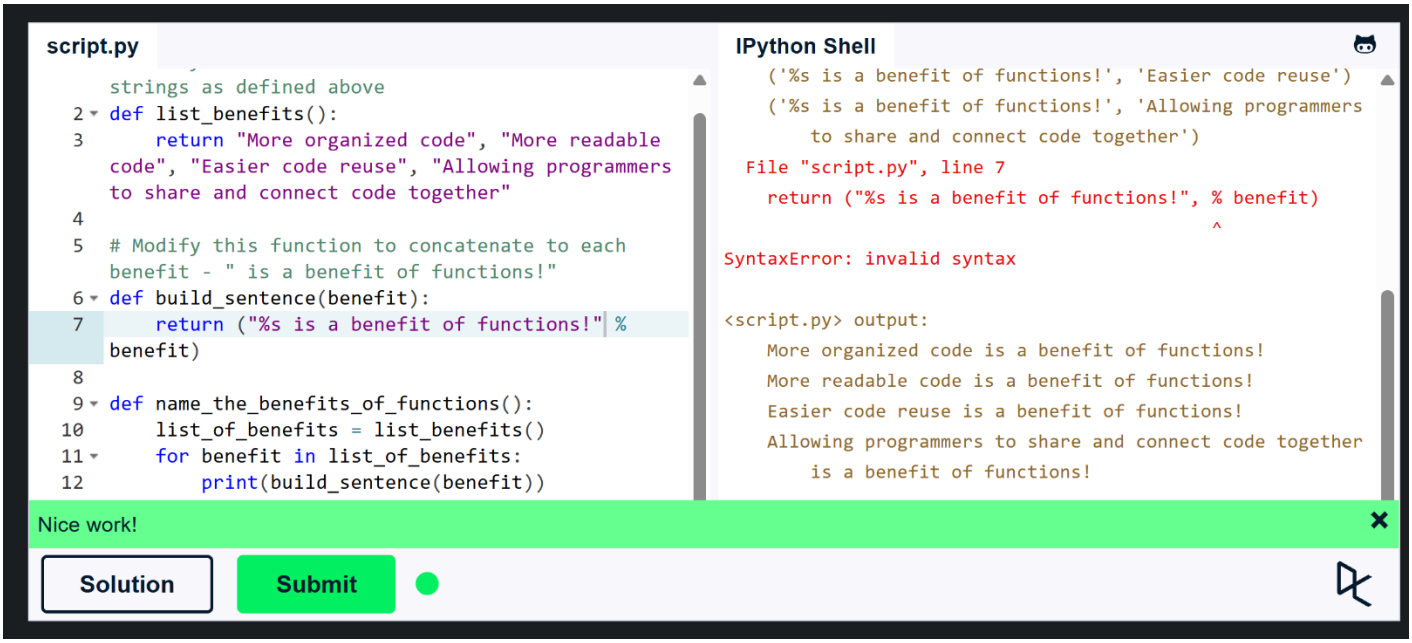

script.py
1 # define the Vehicle class
2 class Vehicle:
3 name = ""
4 kind = "car"
5 color = ""
6 value = 100.00
7 def description(self):
8 desc_str = "%s is a %s %s worth $%.2f." %
(self.name, self.color, self.kind, self.value)
9 return desc_str
10
11 # your code goes here
12 car1 = Vehicle()
13 car1.name = "Fer"
14 car1.color = "red"
15 car1.kind = "convertible"
IPython Shell
<script.py> output:
Fer is a red convertible worth $60000.00.
Jump is a blue van worth $10000.00.
In [1]:
Great job!
Solution
Submit

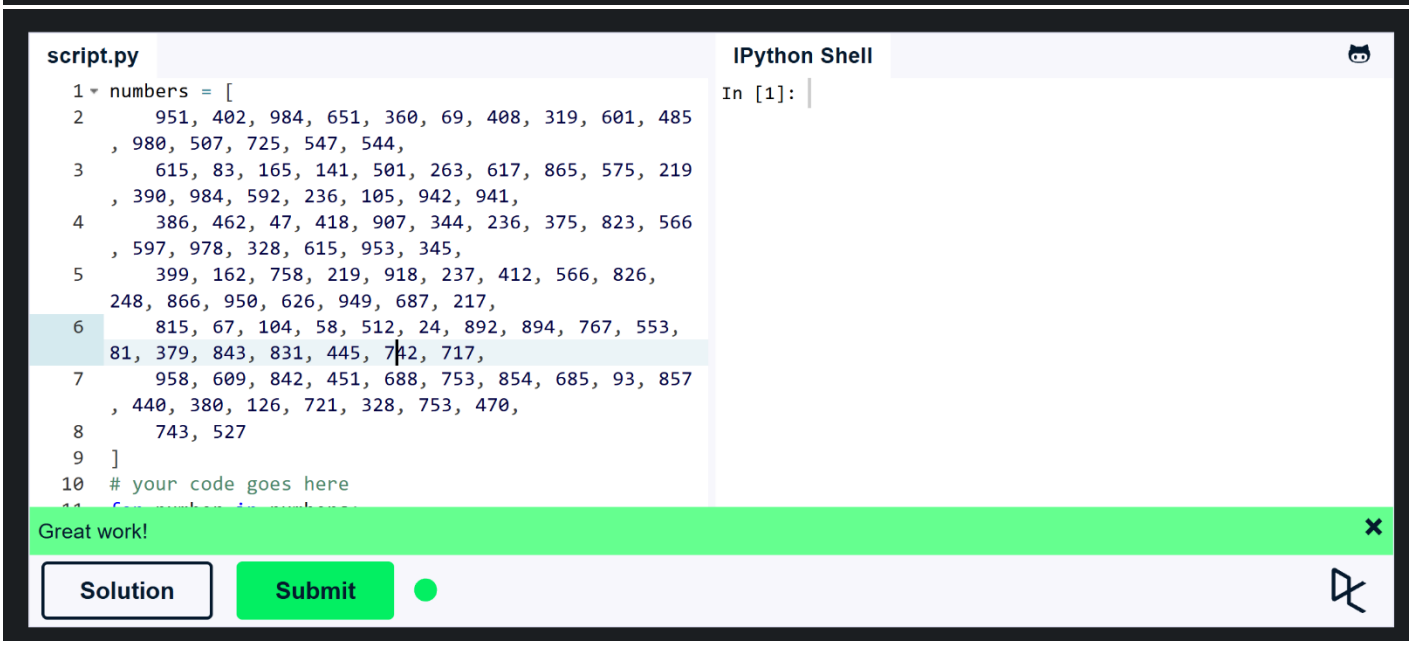

script.py
1 import re
2
3 # Your code goes here
4 find_members = []
5 for member in dir(re):
6 if "find" in member:
7 find_members.append(member)
8
9 print(sorted(find_members))
IPython Shell
<script.py> output:
['findall', 'finditer']
In [1]:
Great work!
Solution
Submit

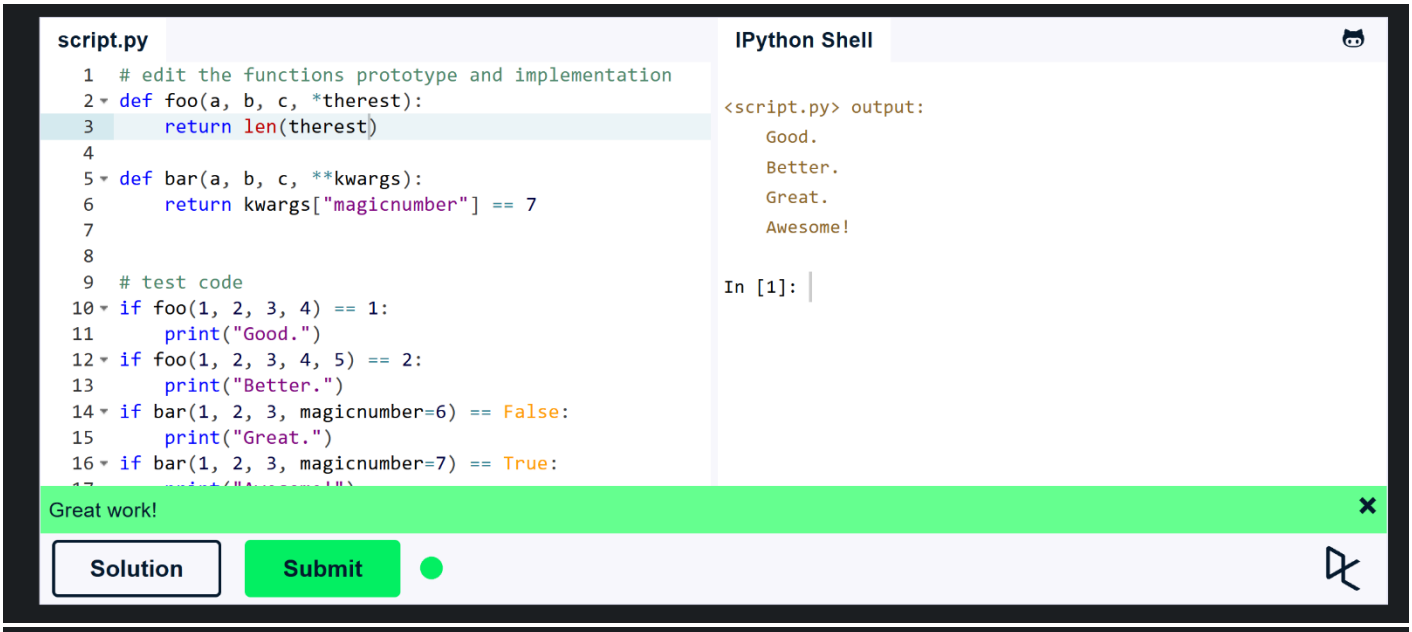

script.py
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
12.7]
2 nr_list = [int(x) for x in numbers if x>0]
3
4 print(nr_list)
IPython Shell
<script.py> output:
[34, 44, 68, 44, 12]
In [1]:
Very nice!
Solution
Submit

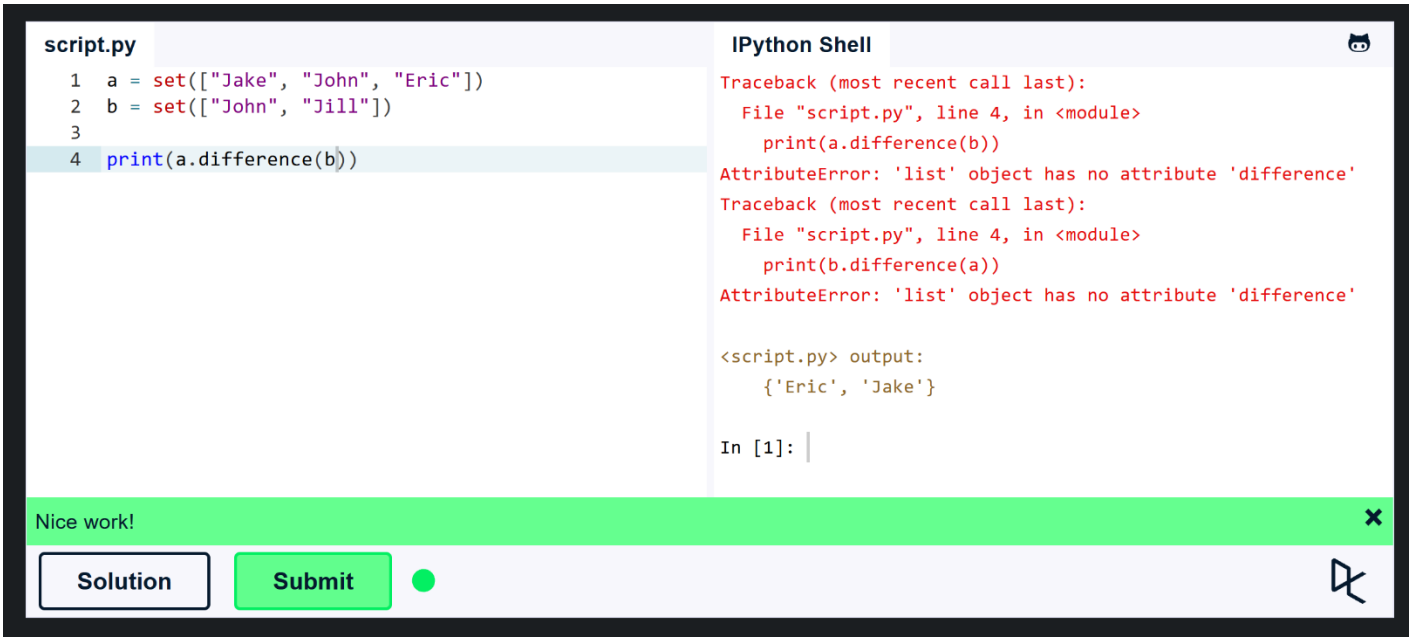

script.py
1 l = [2,4,7,3,14,19]
2 for i in l:
3 check = lambda x : (x%2)
4 if (check(i) == 0):
5 print("False")
6 else: print("True")
IPython Shell
^
SyntaxError: invalid syntax
File "script.py", line 6
else: print("True")
^
SyntaxError: invalid syntax
<script.py> output:
False
False
True
True
False
True
Nice work!
Solution
Submit

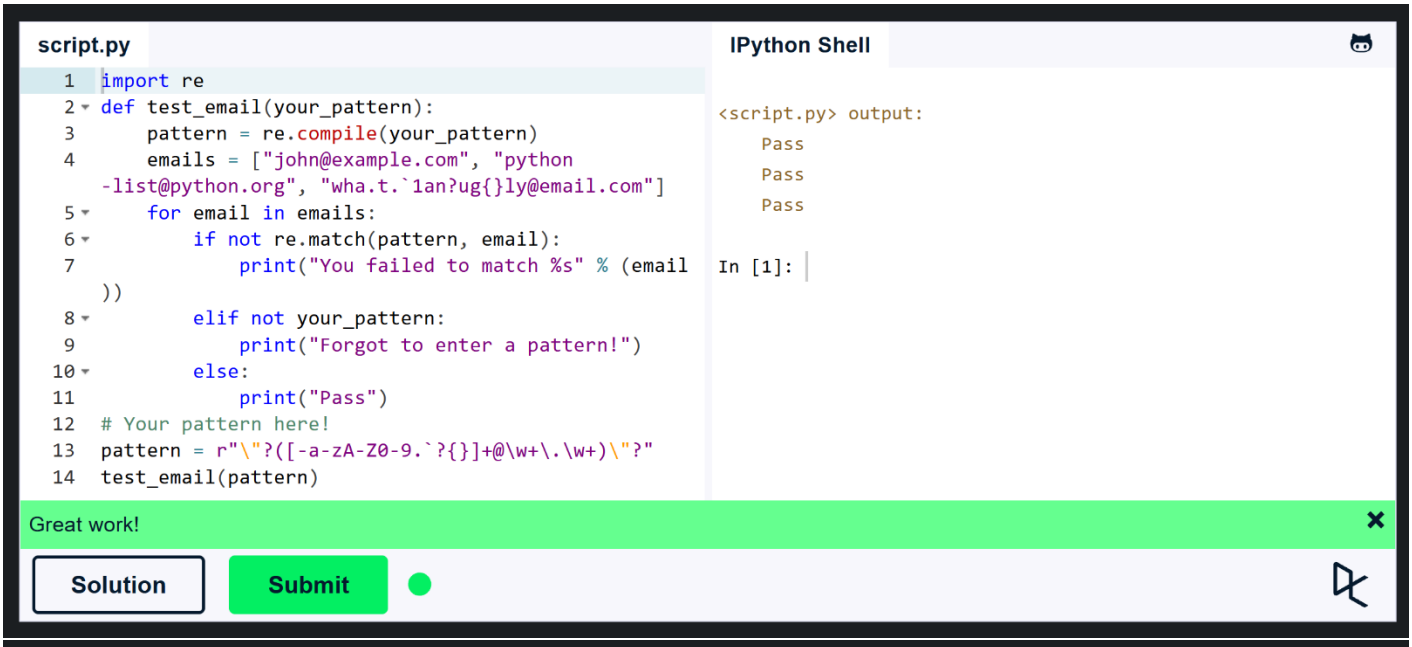

script.py
1 # fill in this function
2 def fib():
3 a,b = 1,1
4 while 1:
5 yield a
6 a,b = b,a + b
7
8 # testing code
9 import types
10 if type(fib()) == types.GeneratorType:
11 print("Good, The fib function is a generator.")
12
13 counter = 0
14 for n in fib():
15 print(n)
16 counter += 1
IPython Shell
<script.py> output:
Good, The fib function is a generator.
1
1
2
3
5
8
13
21
34
55
Good work!
Solution
Submit

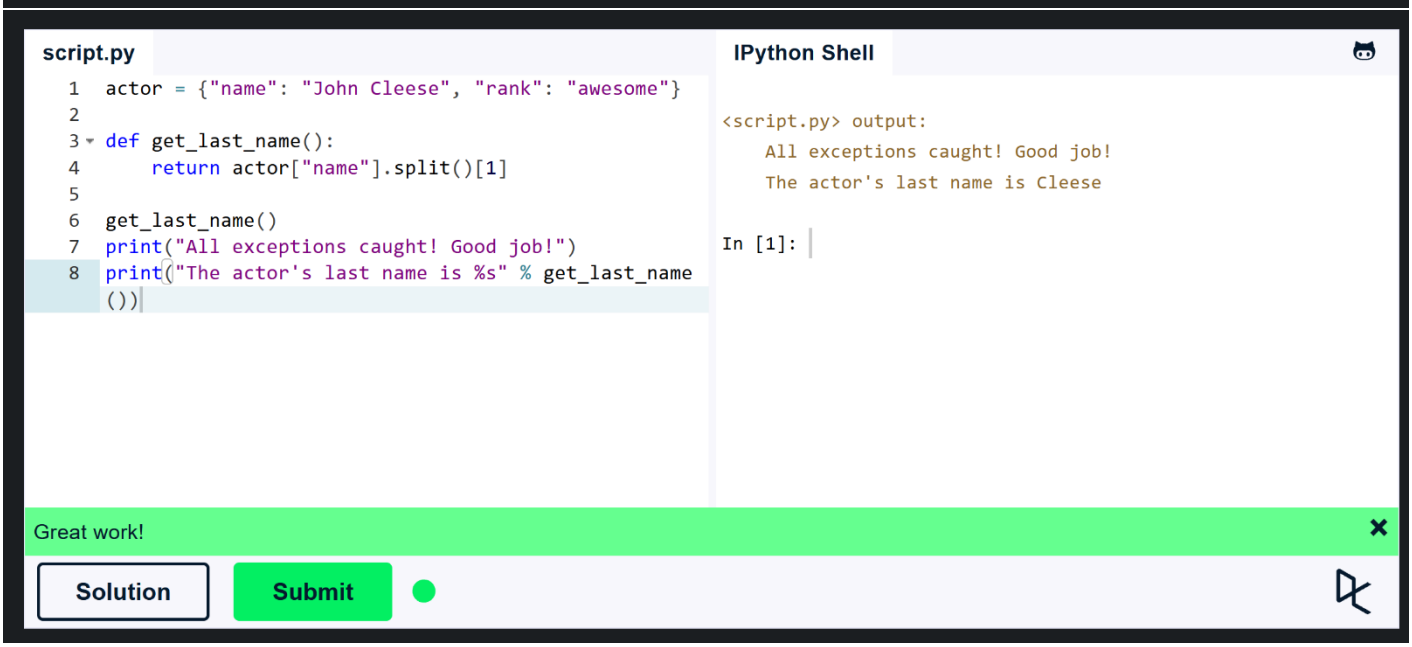

script.py
strings as defined above
def list_benefits():
return "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
Modify this function to concatenate to each benefit - " is a benefit of functions!"
def build_sentence(benefit):
return ("%s is a benefit of functions!" % benefit)
def name_the_benefits_of_functions():
list_of_benefits = list_benefits()
for benefit in list_of_benefits:
print(build_sentence(benefit))
IPython Shell
('%s is a benefit of functions!', 'Easier code reuse')
('%s is a benefit of functions!', 'Allowing programmers to share and connect code together')
File "script.py", line 7
return ("%s is a benefit of functions!", % benefit)
^
SyntaxError: invalid syntax
<script.py> output:
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code together is a benefit of functions!
Nice work!
Solution
Submit

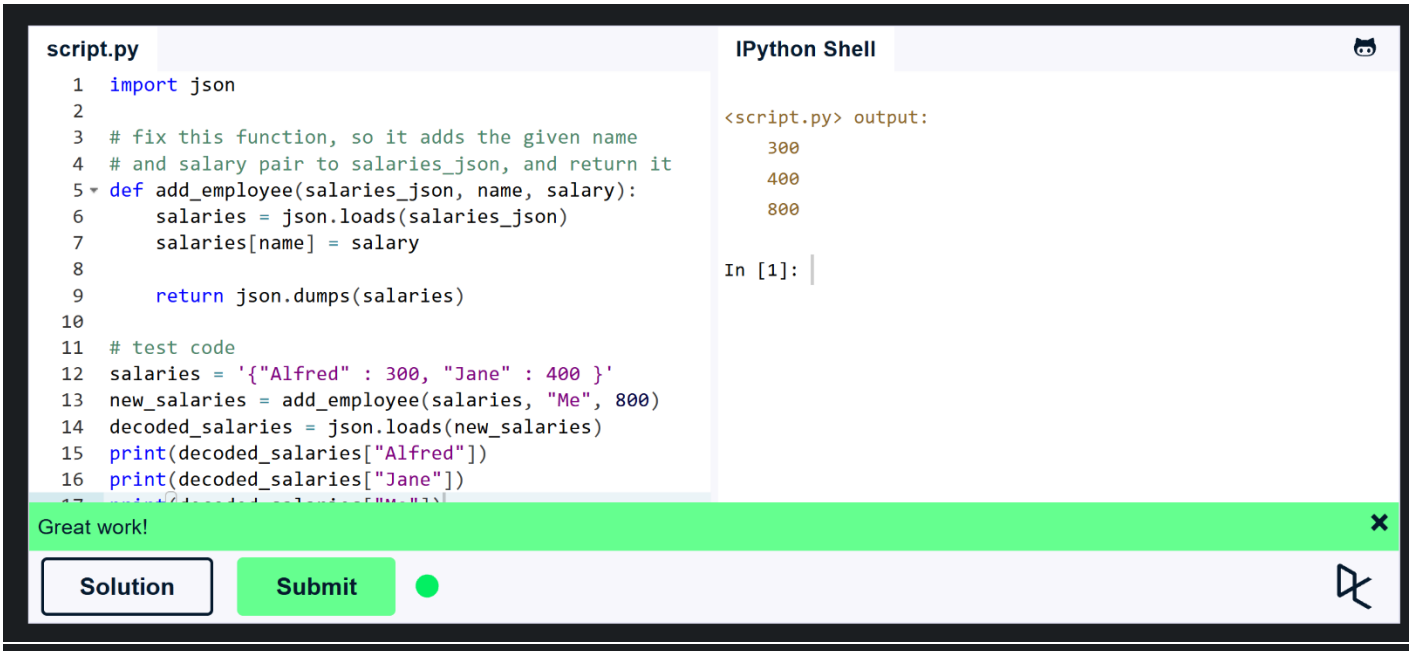

script.py
numbers = [
951, 402, 984, 651, 360, 69, 408, 319, 601, 485, 980, 507, 725, 547, 544,
615, 83, 165, 141, 501, 263, 617, 865, 575, 219, 390, 984, 592, 236, 105, 942, 941,
386, 462, 47, 418, 907, 344, 236, 375, 823, 566, 597, 978, 328, 615, 953, 345,
399, 162, 758, 219, 918, 237, 412, 566, 826, 248, 866, 950, 626, 949, 687, 217,
815, 67, 104, 58, 512, 24, 892, 894, 767, 553, 81, 379, 843, 831, 445, 742, 717,
958, 609, 842, 451, 688, 753, 854, 685, 93, 857, 440, 380, 126, 721, 328, 753, 470,
743, 527
]
your code goes here
IPython Shell
In [1]:
Great work!
Solution
Submit

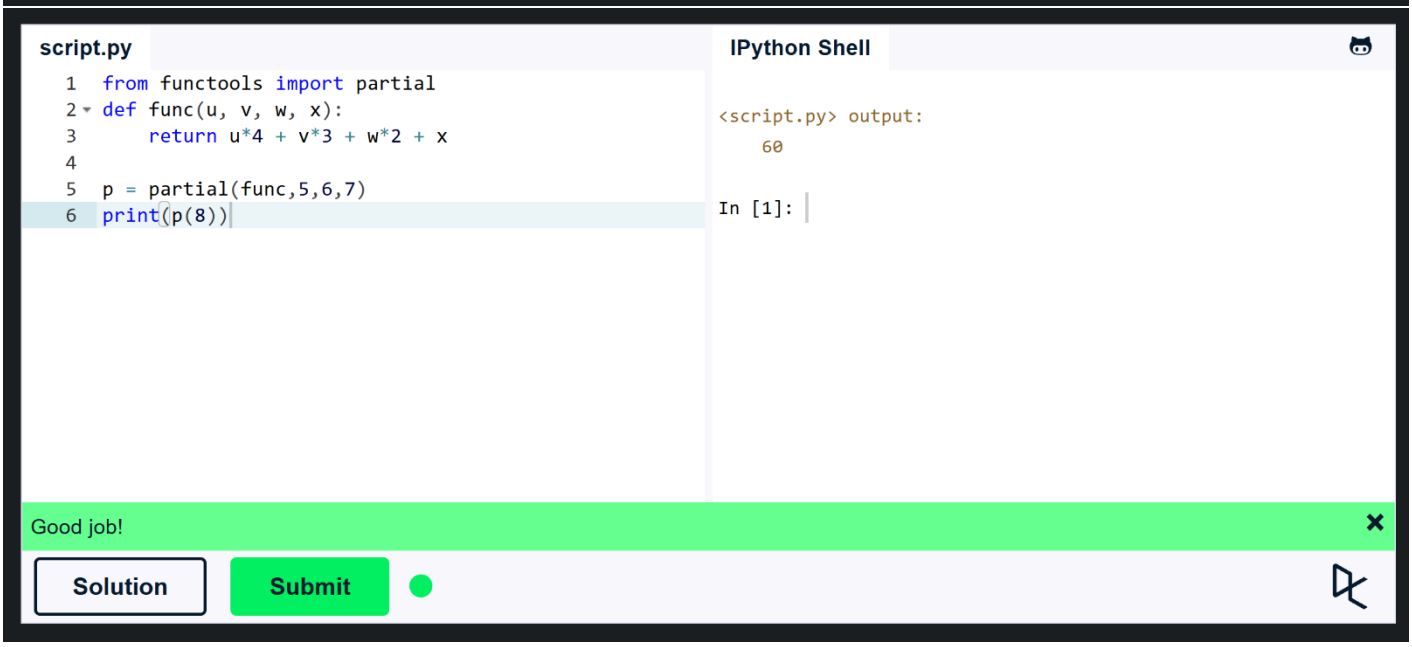

script.py
edit the functions prototype and implementation
def foo(a, b, c, *therest):
return len(therest)
def bar(a, b, c, **kwargs):
return kwargs["magicnumber"] == 7
test code
if foo(1, 2, 3, 4) == 1:
print("Good.")
if foo(1, 2, 3, 4, 5) == 2:
print("Better.")
if bar(1, 2, 3, magicnumber=6) == False:
print("Great.")
if bar(1, 2, 3, magicnumber=7) == True:
IPython Shell
<script.py> output:
Good.
Better.
Great.
Awesome!
In [1]:
Great work!
Solution
Submit


script.py
a = set(["Jake", "John", "Eric"])
b = set(["John", "Jill"])
print(a.difference(b))
IPython Shell
Traceback (most recent call last):
File "script.py", line 4, in <module>
print(a.difference(b))
AttributeError: 'list' object has no attribute 'difference'
Traceback (most recent call last):
File "script.py", line 4, in <module>
print(b.difference(a))
AttributeError: 'list' object has no attribute 'difference'
<script.py> output:
{'Eric', 'Jake'}
In [1]:
Nice work!
Solution
Submit


script.py
1 import re
2 def test_email(your_pattern):
3 pattern = re.compile(your_pattern)
4 emails = ["john@example.com", "python -list@python.org", "wha.t.`1an?ug{}ly@email.com"]
5 for email in emails:
6 if not re.match(pattern, email):
7 print("You failed to match %s" % (email))
8 elif not your_pattern:
9 print("Forgot to enter a pattern!")
10 else:
11 print("Pass")
12 # Your pattern here!
13 pattern = r"\"?([-a-zA-Z0-9.`?{}]+@\w+\.\w+)\"?"
14 test_email(pattern)
IPython Shell
<script.py> output:
Pass
Pass
Pass
In [1]:
Great work!
Solution
Submit


script.py
1 actor = {"name": "John Cleese", "rank": "awesome"}
2
3 def get_last_name():
4 return actor["name"].split()[1]
5
6 get_last_name()
7 print("All exceptions caught! Good job!")
8 print("The actor's last name is %s" % get_last_name())
IPython Shell
<script.py> output:
All exceptions caught! Good job!
The actor's last name is Cleese
In [1]:
Great work!
Solution
Submit





Am practicat fundamente ale limbajului Python pe care le știam, dar am învățat și lucruri noi:

- **Generators**
- **Lamba Functions**
- **Regular Expressions**
- **Serialization**
- **Partial functions**

CONCLUZII

În această lucrare de laborator mi-am adus aminte și am aprofundat noțiunile fundamentale ale limbajului Python pe care le cunoșteam anterior, dar în același timp am întâlnit concepte si practici noi, care au complicat lucrurile, dar care pot fi folositoare pe viitor in practica programării. Am înțeles modul eficient în care Python gestionează colecțiile de date (liste, dicționare, set-uri). Am aprofundat structurarea codului prin programarea orientată pe obiect (Classes, Objects) și modularizarea acestora. Am înâlnit și informație nouă privind tratarea excepțiilor și persistența datelor prin procesul de serializare.

BIBLIOGRAFIE

1. https://www.codedex.io/python
2. https://www.learnpython.org/